

PLANARIAN: Managing Agent State with Statepoints

Jinnan Guo
Imperial College London

Hao (Mark) Chen
Imperial College London

Kapil Vaswani
Indian Institute of Science Bangalore
SPARC

Andrew Paverd
Microsoft Security Response Center

Peter Pietzuch
Imperial College London

Abstract

LLM agents solve complex tasks by iteratively changing files, invoking local tools, and interacting with remote services, which modifies state across their local environment and remote services. Today, agents and users must manage these changes explicitly, whether reverting exploratory actions or recovering from erroneous ones. Doing so safely requires coordinated actions, yet current agent harnesses lack unified abstractions and mechanisms for managing local and remote state consistently and efficiently.

We describe PLANARIAN, an agent runtime with state management that enables agents and users to recover from erroneous actions and explore alternative executions over consistent local and remote environment state. PLANARIAN introduces the abstraction of *agent statepoints*, which are consistent, restorable point-in-time versions of the environment state. PLANARIAN exposes three state-management primitives to agents and users: (i) *snapshot* creates a new statepoint spanning local and remote state without requiring external services to support checkpoints: it relies on efficient incremental process and file system snapshotting to capture local sandboxed state, and transparently records compensating actions to undo remote state changes; (ii) *rollback* restores the environment to a previous statepoint by reverting to a prior local checkpoint and replaying compensating actions for remote state changes; and (iii) *fork* creates multiple isolated branches from a statepoint, enabling the agent to explore alternatives in parallel. We show that PLANARIAN enables agents to undo mistakes and explore alternatives in parallel, improving task quality by up to 15×, and allows users to recover from erroneous actions with only 3% overhead.

1 Introduction

Agents perform tasks that require repeated interaction with a host machine, from modifying software to administering services [3, 4, 15]. An *agent harness* invokes a large language model (LLM) with *context* comprising instructions, prior actions, and tool results. It uses the model’s output to invoke *tools* and incorporates their results into subsequent context. Tool calls request commands or service operations that act on *environment state*: local files and processes, and state held by external services, such as database state. Local tools can execute directly on the host or within a *sandbox* that

isolates their execution. The harness can invoke remote tools through the *model context protocol* (MCP) [2].

When an agent fails to complete a task, it may have to undo earlier actions before trying different steps. A user may also want to reverse an unwanted change, such as the deletion of files or incorrect updates to a remote calendar service. For the agent, reconstructing state requires extra tool calls, which may be costly or impossible if data was lost. We observe that explicit *state management* would let agents and users recover from mistakes without manually having to repair the affected state. Agents could also explore the effect of alternative actions on the environment efficiently without repeating successful work or carrying over unwanted effects.

Supporting such state management alongside existing agent harnesses, however, raises three challenges:

(1) Consistency across local and remote state. State management requires *environment consistency*, i.e., it must restore both local and remote state consistently. Local files and processes can be checkpointed, but MCP endpoints of remote network services may provide neither snapshots nor undo operations. Recovering only local state can result in inconsistencies though: for example, a data-cleaning agent may update database records through an MCP endpoint and write a local report of its changes. Reverting only the local report leaves it inconsistent with the updated remote database.

(2) Integration with agent execution. Users and agents must be able to perform state management through existing agent harnesses. In addition, state management must guarantee *context consistency*, i.e., the agent must be aware of the changes to the environment state. After the environment state is restored, the history of tool calls may describe effects that no longer exist. Simply retaining this history can mislead the agent; discarding it loses evidence of unsuccessful actions that could guide future actions. State management therefore affects both how the harness dispatches tools and what context it supplies.

(3) Overhead of state management. Frequent state management must not dominate task execution time or storage use. Local state, including files and process memory, may be large, even when individual tool calls change little state. State management overheads may accumulate both as execution advances and whenever an earlier state is revisited to try different actions. Managing remote state also incurs communication latency and is constrained by MCP interfaces.

Prior approaches solve only some of these challenges. Checkpointing systems such as DMTCP [18] and Catalyzer [25] preserve local execution state, but cannot account for remote service state or agent decisions. Transactional workflows [45], speculative configuration repair [37], and intrusion recovery with external compensation [30] establish broader recovery models, but target generic workflows or application repair rather than agent exploration. Without environment and context consistency, state management cannot serve the parties that need it: an agent cannot safely roll back on its own, a harness cannot search over alternative branches, and a user cannot reliably undo an unwanted change. The open problem is thus to make an agent’s evolving environment state a unit of recovery and exploration, which is consistent across local and remote state, consistent with the agent’s context, and efficient enough to be used repeatedly.

We describe PLANARIAN, an agent runtime for state management that operates together with existing agent harnesses. PLANARIAN provides a unified abstraction for managing local and remote environment state, enabling consistent recovery and exploration while supplying the information needed for subsequent agent decisions. For this, PLANARIAN makes three technical contributions:

(1) Agent statepoints and primitives. Recovery and exploration need a common abstraction for managing both local and remote state. PLANARIAN introduces *agent statepoints*: a statepoint captures both local files and running processes together with the remote state changed through MCP operations. A statepoint remains restorable even when the remote service cannot take checkpoints, and carries a description of the captured state and the outcomes of prior executions. For state management, three primitives operate over statepoints: (i) *snapshot* records a new statepoint; (ii) *rollback* returns to a statepoint to undo state changes; and (iii) *fork* starts an isolated alternative execution in a branch from a statepoint, while preserving the existing state.

(2) Integrating agent statepoints with agent execution. PLANARIAN exposes its state-management primitives (a) as tools that agents invoke directly, or through APIs for (b) harness-driven search or (c) manual user requests. During execution, the agent must select suitable statepoints and interpret them correctly. PLANARIAN uses the evidence associated with statepoints both to inform this choice and to update the agent context afterward. The agent harness retains responsibility for LLM invocations and context construction, routing tool calls through PLANARIAN and incorporating the returned results. PLANARIAN manages environment state through local tool sandboxing and MCP proxying, and captures statepoints between tool calls. After restoration, PLANARIAN supplies the selected state’s description and prior outcomes to the agent harness to append to the context. This identifies the environment state now available to agents without discarding the information needed to choose different actions.

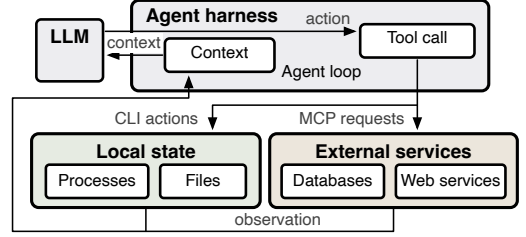


Figure 1. Agent harness invoking the LLM and tools

(3) Local checkpointing and remote compensation. To keep the overhead of state management low, PLANARIAN limits the state that must be copied locally and reconstructed remotely when creating or reverting to statepoints. For *local state*, PLANARIAN combines incremental copy-on-write filesystem snapshots [17] with CRIU process checkpoints [7] that record only changed memory. PLANARIAN re-establishes the restored checkpoint as the baseline for subsequent memory changes, allowing incremental capture across rollback and fork without modifying CRIU or the kernel. This avoids repeatedly copying unchanged memory during exploration. For *remote state*, PLANARIAN proxies supported MCP requests so that their effects can be reversed through *compensating actions*—service operations that undo earlier changes. For example, a database MCP proxy preserves original database row values and affected keys within the transaction that modifies them, allowing destructive updates to be undone. We evaluate PLANARIAN with agents in coding, personal assistance, system administration, database management, and gaming tasks. In the Mario platform game, an agent that uses PLANARIAN to retry from statepoints achieves 15× the game score of an agent without state management. When solving system-administration tasks with MCTS-style tree search, PLANARIAN takes snapshots 10× faster than a baseline that uses full process and filesystem checkpoints. Across system and database administration tasks, state management adds only 1%–3% to completion time, including coordinated recovery for an agent that modifies a remote database through MCP. These results show that PLANARIAN enables agents to revisit decisions and explore alternatives while incurring little overhead for state management.

2 State Management for Agents

First we describe agent execution (§2.1), establish the requirements for managing environment state (§2.2), and assess existing state-management approaches (§2.3).

2.1 Agent execution

As LLMs become capable of using tools, users delegate everyday tasks, such as coding, system administration and data maintenance, to agents for automation [33, 39].

As shown in Fig. 1, an *agent* uses an LLM to choose actions toward a task objective. An *agent harness* coordinates the model and tool invocations through which the agent interacts with this environment through an agent loop [43]:

it invokes the LLM with context, interprets the output to invoke tools, and incorporates tool results into the context for subsequent LLM invocations. The agent’s *environment state* thus comprises local machine state, such as files and running processes, and external network service state.

Agent harness implementations range from general ones (e.g., LangChain [5] and AutoGen [41]) to specialized harnesses for coding agents (e.g., Claude Code [3], Codex [4]) and personal assistants (e.g., OpenClaw [15], Nanobot [13]).

Tools provide access to local commands and files and remote service operations. An agent harness can invoke local tools on the host and remote tools through the model context protocol (MCP) [2]. MCP provides a common interface for invoking APIs exposed by remote servers.

Sandboxing [36] is an optional safety feature that confines local tool execution to an isolated environment. Restricting access to host files and processes limits the damage that erroneous commands can cause outside the sandbox. Sandboxes can be realized using containers [31], which isolate processes and their filesystem view through OS mechanisms, such as Linux namespaces. A harness creates and starts a container from an image containing the required software, executes local tool calls inside it, and removes it when the task ends. Sandboxing libraries and SDKs, such as E2B [12], provide APIs through which harnesses can use sandboxing.

2.2 Managing environment state

As an agent executes, it mutates the environment state, producing visible side effects on the machine. When an agent updates environment state, there are two broad requirements: **Effectiveness.** Agents often solve challenging tasks through search and exploration [44]. By sampling multiple rollouts from an LLM, an agent can improve its problem-solving capability by allowing it to try alternative plans and learn from unsuccessful attempts. Such exploration mechanisms are increasingly common: *pass@K* allows an agent to attempt a task *K* times [22]; Monte Carlo tree search (MCTS) [38] performs exploration and backtracking as part of the harness; and EAPO [27] encourages the agent to explore and recover from mistakes through backtracking.

At the system level, an agent must be able to explore plans sequentially or concurrently, with some inevitably failing. These approaches impact the environment state, as they require backtracking and state branching so that side effects of old plans do not affect subsequent execution.

Safety. Since LLM outputs are non-deterministic and unreliable (cf. hallucination [28], sycophancy [26]), an agent’s tool calls can produce unintended, harmful side effects, e.g., deleting required files, installing incorrect packages, or committing incorrect writes to a database. Unlike a conventional program, an agent writes and executes commands without human review, so it may break a production environment before the mistake is noticed by the user. Running an agent

safely thus requires the ability to *rollback* the state after a mistake, preserving the integrity of the environment state.

We observe that supporting both effectiveness and safety requires *state management* that enables exploration of alternative executions and recovery from unwanted changes: exploration allows alternative actions to be tried without their effects interfering, while recovery undoes unwanted effects. State management must cover local and remote state, and maintain consistency across these types of state and with the agent’s context. Furthermore, recovery and exploration should be accessible to users, harnesses, and agents: a user may notice a mistake, a harness may organize a search, or an agent may decide to retry an action. Since each acts on the same environment, the guarantees must hold regardless of who initiates the operation. These state management capabilities must also be implemented efficiently to not degrade the performance of agent execution.

2.3 Existing state management approaches

Prior approaches only support some of these requirements for state management. Tab. 1 compares approaches along four dimensions: the *environment state* columns describe whether local files, processes and remote service state are managed; the *consistency* column refers to coordination between local and remote state (L/R), and between agent context and environment state (C/E); the *primitives* column considers support for recording, restoring, and branching versions of the environment state; and the *stakeholders* column identifies whether users, harness code, or agents can invoke state management.

Framework checkpointing. Systems such as LangChain [5] and OpenClaw [15] offer *framework checkpoints* that preserve conversation history, intermediate results, and workflow progress. In terms of *environment state*, this covers at most parts of the local state and excludes remote state. Their *primitives* resume or revisit workflow steps, rather than restoring and branching the complete environment. Consequently, *consistency* between context and environment is limited: rewinding a conversation does not undo a database update, and restoring workspace files does not restore the processes using them. For *stakeholders*, these approaches support user- or harness-initiated recovery: orchestration code selects a saved workflow state and resumes execution from it. This does not itself provide agent-driven recovery, which would require exposing recovery choices to the LLM and interpreting its selection within the running agent loop. **Sandbox checkpointing.** Approaches such as Podman [16], E2B [12], CubeSandbox [8], and DeltaBox [24] provide *sandbox checkpoints*. Their *environment state* includes local files and processes, and their *primitives* support snapshot, rollback, and branching through user and harness interfaces. Some also preserve context with local state, providing C/E *consistency*, and recent systems reduce checkpointing costs

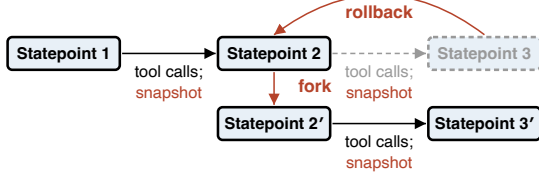


Figure 3. State management primitives for agent statepoints

state at a single point in time. Otherwise, independently captured local and remote state could describe an environment that never existed. Treating them as one version makes consistency a requirement of the abstraction, rather than a responsibility of each caller.

Each statepoint also carries *state evidence* that gives the environment version meaning: a description of its state and the outcomes of executions that previously proceeded from it. The evidence helps stakeholders (e.g., the agent or user) select an appropriate statepoint to restore and avoid repeating executions that already failed. Evidence is associated metadata, distinct from both the captured environment state and the agent’s LLM context. The evidence can be extended with a summary of what an execution attempted after that statepoint and what happened, such as completing the task or encountering an error. Adding this information leaves the recorded environment version unchanged. Keeping the evidence separate from the environment state allows knowledge of unsuccessful attempts to guide subsequent decisions even after returning to an earlier version.

3.2 Primitives

The agent statepoint enables three state-management primitives (see Fig. 3): *snapshot*, *rollback*, and *fork*, which meet the recovery and exploration requirements in §2.2. They operate on consistent environment versions and their associated state evidence, giving users, harnesses, and agents the same guarantees without requiring each caller to coordinate local and remote state.

Snapshot. To return to an earlier execution point requires that the state was preserved consistently. *Snapshot* captures both local and remote state and generates a new statepoint. It records the environment at a chosen boundary between tool calls as one restorable version, together with a state description. In Fig. 3, each transition from statepoint 1 to 2 and from 2 to 3 comprises one or more tool calls followed by an explicit snapshot; note that the abstraction does not require a snapshot after every call. Each statepoint covers local files and running processes as well as the affected remote service state, even when the service provides no checkpoint API.

Rollback. Recovering from an error or abandoning an unsuccessful attempt requires undoing its effects across both state domains. *Rollback* restores the environment state to a previously snapshotted statepoint. In Fig. 3, returning from statepoint 3 to 2 abandons the later version shown in gray. For example, an agent records statepoint 2 after installing

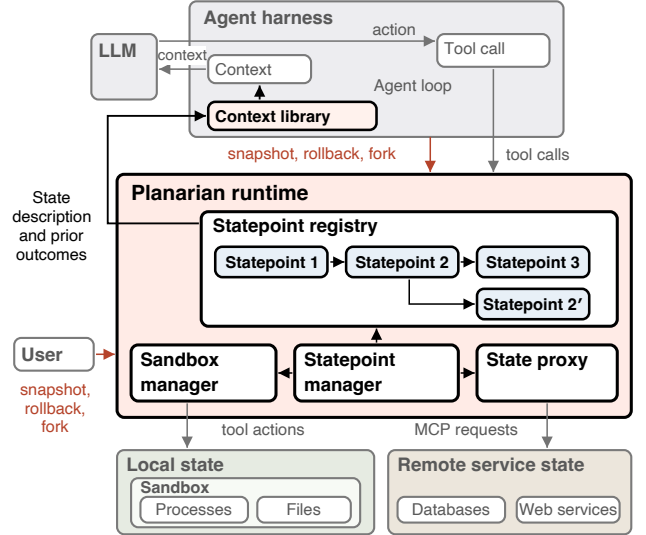


Figure 4. Design of PLANARIAN

dependencies, then creates a file and registers it in a database. Rollback must undo both the file creation and the database update while preserving the dependencies. The state evidence helps callers choose where to return and identifies the restored environment. The agent’s context is not rewound, and it retains knowledge of unsuccessful attempts [46].

Fork. If the agent explores, the environment state of each attempt must remain isolated. *Fork* creates a new environment branch from a previously captured statepoint. In Fig. 3, statepoint 2’ branches from statepoint 2; alternative tool calls followed by a snapshot produce statepoint 3’. Fork preserves the source branch, allowing it to continue independently. Local files and processes are isolated between branches; extending this isolation to remote state requires the service to support independent branches. The associated state evidence describes the common starting point and prior attempts, helping callers choose alternative actions.

4 PLANARIAN Design

PLANARIAN is an agent runtime designed to realize the agent statepoint abstraction and state management primitives in §3. In this section, we describe the overall design of PLANARIAN, and explain how it enables state management for agents. After that, §5 and §6 provide specific implementation details of how PLANARIAN manages local and remote state, respectively.

4.1 Overview

Fig. 4 shows PLANARIAN’s architectural components. During agentic execution, the state management functionality provided by PLANARIAN can be invoked by the user, the harness, or the agent itself. The harness routes tool execution through PLANARIAN and integrates with PLANARIAN via a context library. Tab. 2 shows the state-management primitives PLANARIAN provides. A caller uses *snapshot* to record a

statepoint, rollback to return the sandbox and remote state to that version, or the local fork operation to create another sandbox from its local state. The sandbox handle *sb* identifies the local execution environment, and *sp* identifies a statepoint, whose local and remote parts of state are *sp-local* and *sp-remote*. The capture and compensate hooks connect remote-service recovery to snapshot and rollback; the statepoint manager coordinates them with the sandbox operations.

4.2 Local state management

To enable state management, all tool invocations that read or mutate *local* state (e.g., processes, filesystems, etc.) must be run within a sandbox. This is already considered best practice for agentic systems in order to isolate the agent’s execution from the host environment.

In PLANARIAN, a *sandbox manager* creates and manages one or more sandboxes and dispatches the agent’s tool calls to the appropriate sandbox. When a user, harness, or agent requests a snapshot, the sandbox manager captures the local state at a boundary between tool invocations. Callers can request a snapshot after each call or after several calls; the primitive does not prescribe a fixed frequency.

Delegating snapshot execution to the sandbox manager separates this choice from the mechanics of managing sandboxes and capturing their state. PLANARIAN uses Podman-based sandboxes [16], which combine incremental process checkpointing with zero-copy filesystem snapshots to reduce latency and copying overhead (§5). Snapshot and fork can also overlap LLM inference (§6.3), which can hide latency (§7.2).

4.3 Remote state management

For remote services without native versioning, PLANARIAN relies on *compensating actions* that undo supported operations without requiring a service checkpoint. To achieve this, all tool calls that mutate remote state are routed through the *state proxy*—a transparent interception layer between the agent harness and the target MCP server (§6.1). For each supported tool call, the state proxy transforms the original request into a compensable form and records the information used to generate compensating actions. Requests that cannot be made compensable are rejected before execution.

When a statepoint is created, the state proxy records its current position in the sequence of MCP calls. On a `rollback(sp, sb, compensate)` request, it uses the recorded MCP call position to identify the subsequent actions and applies their compensating actions in reverse order.

4.4 State consistency

To fully realize the statepoint abstraction (contribution #1), PLANARIAN must ensure that each statepoint is a consistent view of both the agent’s local and remote state. This

Table 2. PLANARIAN’s state management APIs

Function	Description
<code>snapshot(sb, capture) → sp</code>	Records statepoint <i>sp</i>
<code>rollback(sp, sb, compensate)</code>	Restores sandbox, service
<code>fork(sp-local, sb) → sb'</code>	Forks sandbox from <i>sp-local</i>
<code>capture(sb, service) → sp-remote</code>	Hook: records remote state
<code>compensate(sp-remote, sb, service)</code>	Hook: restores service

is handled by the *statepoint manager* component, which coordinates between the sandbox manager and state proxy components. To create a new statepoint, it first freezes the sandbox, and takes the sandbox snapshot. After that, the MCP call position is captured from the state proxy. Once both parts are complete, the sandbox can be resumed.

During a snapshot, the statepoint manager holds an exclusive lock: it waits for any in-flight commands and MCP calls to finish before commencing the snapshot, and new calls are blocked until the snapshot completes (§6.3). The local and remote state are combined into a single statepoint, representing a point-in-time version of the agent’s environment state, which is committed to the statepoint registry (§6.2).

Rolling back to a statepoint is also a two-step process: (i) the statepoint manager applies the compensating actions through the state proxy, bringing the remote state to its logically equivalent point; and (ii) it restores the local state of the sandbox from the snapshot.

The local `fork(sp-local, sb)` operation creates an isolated sandbox from a statepoint’s local state, preserving the original sandbox. The local checkpointing mechanisms in §5 support subsequent snapshots and rollbacks within each branch. Extending a branch to remote state requires service-side branching, e.g., from a database such as Neon [14] or Dolt [11]. When a service supports compensation only, the forked sandbox cannot use that state proxy, preventing interference with the source branch (§6.2).

4.5 Agent context management

To integrate the statepoint primitives into the agent’s execution (contribution #2), the necessary information must be brought into the agent’s context. In PLANARIAN, a *context library* collects state evidence and supplies information about state-management operations to the harness. It makes evidence available to users, harnesses, and agents when choosing a statepoint for rollback, and supplies the selected state’s description and prior outcomes after rollback. The harness retains responsibility for LLM invocation and context construction, including incorporating this information into the agent’s context. It exposes several APIs to the harness, as shown in Tab. 3:

Snapshot creation. The context library gathers state from the harness, e.g., the progress of a task, the player information from a game, or the state of the sandbox. This constructs a mapping between a statepoint and its state, making

Table 3. PLANARIAN’s context management APIs

Action	Context API	Description
snapshot	StateRecord(...)	Records state description
	AppendOutcome(...)	Appends outcomes
selection	ConstructLedger(...)	Exposes state evidence
rollback	RestoreContext(...)	Supplies rollback context

each statepoint self-describing. However, a static view of the agent’s state at the time of statepoint creation would be insufficient. For example, the state at that point may appear to be suitable for restoration, but it might lead to a deadlock or an undesirable outcome if restored. Therefore, PLANARIAN also appends the execution outcomes that follow this statepoint, recording what previous executions did after this point. The description and the outcomes form the state evidence (§3.1), which guides the choice of statepoint for rollback. Fig. 5 shows the data recorded for each statepoint.

Statepoint selection. Before a caller chooses a statepoint for rollback, the context library exposes the *state ledger*: the evidence associated with committed statepoints in the registry. Users and harnesses can inspect this evidence, and the harness can include it in the LLM’s context so that the agent can choose a suitable statepoint. Once rollback completes, the context library supplies the harness with the selected state’s description and prior execution outcomes, together with notification that rollback has taken place. This information forms the *restore context* returned by `RestoreContext`, which the harness appends to the context. By design, this operation does not remove existing context, so as not to interfere with the harness’s own context management, e.g., context compaction and memory. Without the restore context, the agent might make decisions based on outdated or inconsistent context, leading to repeated mistakes.

5 Local state management

PLANARIAN is implemented as a harness-agnostic runtime that integrates with agent harnesses by routing tool calls through the PLANARIAN APIs. It uses Podman as the agent sandbox (§5.1), where the process tree is checkpointed using CRIU and the file system is snapshotted via ZFS (§5.2). PLANARIAN introduces a process checkpoint protocol that keeps CRIU’s checkpoints incremental across restores (§5.3).

5.1 Sandbox management

PLANARIAN provides a harness-agnostic sandbox manager that manages the lifecycle of the agent sandbox, including launching sandboxes, executing commands, and performing snapshot, rollback, and fork operations. For agents that do not have a sandboxed execution environment, we integrate Podman as the sandbox backend, since it is a security-hardened OCI container engine with first-class checkpoint

support. The sandbox manager launches a long-running Podman container with its built-in isolation primitives, e.g., namespaces, cgroups, system call filtering, and access control, ensuring that an untrusted or erroneous agentic command cannot escape from the sandbox. PLANARIAN uses the Podman container as the boundary of the local state. Each container’s root file system is mounted to a PLANARIAN-managed ZFS [17] dataset, which is a block-level copy-on-write (CoW) filesystem.

5.2 Sandbox snapshot and the local state

The local state of a statepoint contains two parts: a process checkpoint and a file system snapshot. The container’s processes form a process tree rooted at the container’s PID 1. PLANARIAN uses CRIU [7], a process checkpoint tool, to checkpoint the agent container’s process tree into an artifact. This enables PLANARIAN to restore all application processes running at the time of the snapshot. PLANARIAN also snapshots the container’s root file system, which on ZFS is a zero-copy, metadata-only operation. Together, they form a complete restore point of the sandbox. Using this mechanism, PLANARIAN can roll back the file system and the applications running inside it to a clean state, making the sandbox recoverable by preserving its state rather than reconstructing it.

5.3 Continuous incremental checkpointing

To minimize the latency and storage cost of snapshots, PLANARIAN attempts to capture only the incremental changes since the last snapshot. CRIU already supports incremental checkpoints, but with one notable limitation: the first checkpoint after a restore must always be a full (i.e., non-incremental) checkpoint. There are two technical reasons for this: (1) CRIU’s incremental checkpointing relies on the kernel’s soft-dirty bits. After restoring from a checkpoint, the process will have a new address space and new page table entries, thus losing incremental memory tracking (i.e., all the soft-dirty bits will be set). (2) CRIU has an internal safety mechanism based on checkpoint creation time. This mechanism is designed to prevent accidentally creating an incremental checkpoint based on a parent checkpoint from a different process, even if the process ID (PID) is the same (e.g., through PID reuse).¹ Specifically, CRIU falls back to a full dump of a process whose start time is not earlier than the parent checkpoint’s creation time, which prevents incremental checkpoints after a restore.

Since agentic workloads perform very frequent snapshot and restore operations during exploration, creating a full

¹For example, a process forked after a dump inherits the clean soft-dirty bits of the process it forked from, and it may reuse the PID of another checkpointed process that has since exited. The next dump only records the subsequently dirtied pages. When restoring from this incremental dump, CRIU would treat the exited process’s checkpoint as the parent checkpoint of the forked process and restore the forked process with the correct dirty pages but an unrelated base memory state, resulting in incorrect memory.

checkpoint after each restore becomes very costly. Inspired by prior discussions in [1], PLANARIAN proposes an incremental checkpoint protocol that requires no changes to the existing checkpoint/restore (C/R) stack (i.e., CRIU), but enables efficient incremental CRIU checkpointing across restores. The full protocol is shown in Algorithm 1.

Firstly, PLANARIAN uses the insight that a freshly restored process should not have any of the dirty bits set because it has been reconstructed directly from a valid checkpoint. PLANARIAN therefore uses CRIU’s pre-resume hook (line 9), which runs after the process tree has been restored but before execution resumes, to clear the soft-dirty bits of the restored processes (line 12).

Secondly, since PLANARIAN can guarantee that the restored process is always a continuation of the checkpointed process, it can safely bypass CRIU’s checkpoint creation time safety mechanism. Specifically, in the CRIU pre-resume hook (line 14), PLANARIAN computes the epoch time E as one tick after the latest start time of all restored threads. It then waits until the system uptime reaches or exceeds E (line 15), making E a boundary between the restored processes and any process created afterward. After the restore has completed, PLANARIAN modifies the creation time of the parent checkpoint to be the epoch time E (i.e., so that it appears the parent checkpoint was created *after* the current process had started). This enables only the restored process to safely pass the checkpoint creation time check and create incremental checkpoints after the restore. If the restore was part of PLANARIAN’s fork primitive, PLANARIAN does not modify the shared restored image in place, as this could lead to two sandboxes writing the same image concurrently. Instead, PLANARIAN writes a per-sandbox copy of the restored image.

6 Remote state management and consistency

Beyond the local sandbox state, PLANARIAN also captures the remote state (§6.1), managing these heterogeneous states consistently via statepoints using the state management primitives (§6.2). PLANARIAN also provides asynchronous management of snapshot and fork, hiding the state management overhead under the LLM latency (§6.3).

6.1 Remote state proxy

To make remote state restorable, PLANARIAN routes external tool calls through its state proxy component. This component provides similar functionality to the sandbox manager in that it is responsible for capturing and restoring the remote part of a statepoint. When the agent interacts with an external service that supports versioning, e.g., a branchable database [11, 14], the state proxy can simply use the service’s existing functionality to create agent statepoints and roll back or fork from them. However, PLANARIAN also targets the more general case, where the external service does *not* support versioning, and some of its operations are not compensable in their original form. To handle this, the

Algorithm 1 Continuous incremental checkpoint/restore

Input: sandbox sb , checkpoint chain $C = \langle c_0, \dots, c_n \rangle$, full-dump interval k , restore target $c_r \in C$, primitive smp .

```

1: CHECKPOINT( $sb, C, k$ )
2: if  $C = \emptyset \vee |C| \geq k$  then
3:    $c_{n+1} \leftarrow \text{DUMP}(sb, \text{track-mem}); C \leftarrow \langle c_{n+1} \rangle$ 
4: else
5:    $c_{n+1} \leftarrow \text{DUMP}(sb, \text{track-mem}, \text{parent}=c_n); C \leftarrow C \cdot c_{n+1}$ 
6: end if

7: RESTORE( $sb, C, c_r, smp$ )
8: ASSEMBLE( $\langle c_0, \dots, c_r \rangle$ ) from  $C$ ; if  $smp = \text{rollback}$ : STOP( $sb$ )
9:  $sb \leftarrow \text{RESTORE}(sb, c_r, \text{hook}=\text{PRE-RESUME})$ 
10: begin hook PRE-RESUME( $sb$ )
11:   for all  $pid \in \text{PROCESSES}(sb)$  do
12:     WRITE( $/\text{proc}/pid/\text{clear\_refs}$ , 4)
13:   end for
14:    $E \leftarrow 1 + \max\{t.\text{start} : t \in \text{THREADS}(sb)\}$ 
15:   wait until UPTIME()  $\geq E$ ; return  $E$ 
16: end hook
17: if  $smp = \text{rollback}$  then
18:    $c_r.\text{dump\_time} \leftarrow E; C \leftarrow \langle c_0, \dots, c_r \rangle$ 
19: else if  $smp = \text{fork}$  then
20:    $c'_r \leftarrow \text{COPY}(c_r)$ 
21:    $c'_r.\text{dump\_time} \leftarrow E; C \leftarrow \langle c_0, \dots, c_{r-1}, c'_r \rangle$ 
22: end if

```

state proxy first transforms each supported external tool call into a recoverable form and generates a compensating action, which can be used to *undo* the original action.

Proxy integration. To integrate the state proxy with an agent, the agent harness configures the state proxy as the MCP server in place of the target service’s MCP server. When the agent issues an MCP call, the request is dispatched from the harness’s MCP client to the state proxy. This proxy is both an MCP server (to the agent harness) and an MCP client (to the target MCP server), transparently intercepting all requests without changing the original MCP APIs.

Request transformation. Upon receiving a request, the proxy first identifies whether the request can be compensated after transformation; unsupported requests that cannot be compensated are rejected before execution. The proxy then transforms the request into a compensable form. The rewrite and compensation logic is specific to the target MCP server. In PLANARIAN, we prototype the state proxy to support compensable SQL operations for the database MCP server. The protocol transforms a single SQL statement into a compensable form: a pre-image statement captures the necessary table state, the original statement mutates the database state, and a post-image statement records the keys affected by the original statement. These three statements are executed as a single SQL transaction, ensuring atomic execution. This makes the original statement compensable: deleted or modified rows can be restored by applying the

```

1 // --- statepoint metadata --- //
2 "statepoint-id": "...",
3 "parent-statepoint-id": "...",
4 "branch-id": "...",
5 "timestamp": "...",
6 "name": "...",
7 "state-evidence": {
8   "state-description": "...",
9   "prior-outcomes": "...",
10 },
11 // --- statepoint artifacts --- //
12 "local-state": {
13   "criu-checkpoint-path": "...",
14   "zfs-snapshot": "...",
15 },
16 "remote-state": {
17   "external-endpoint": "...",
18   "mcp-call-position": "...",
19   "hook-path": "...",
20 },
21 "status": "discarded | pending | committed"

```

Figure 5. Example entry of an agent statepoint.

values in the pre-image table, and added rows can be deleted according to the keys recorded in the post-image table.

Compensation generation and enforcement. After rewriting, the state proxy generates the compensating action, forwards the rewritten request to the remote target MCP server, and records the compensating action in the undo log after successful execution. All proxy operations are executed under a proxy-level lock, ensuring that requests are serialized and recorded in a request log, so that subsequent compensating actions can be applied in the exact reverse order of their MCP requests in the log.

6.2 Statepoint manager and registry

The statepoint registry manages a collection of the statepoints defined in §3.1. Each registry entry (Fig. 5) includes all three artifacts in a single statepoint: CRIU’s incremental process checkpoint, the zero-copy ZFS snapshot, and the remote state tracked by the state proxy.

To ensure each statepoint is a consistent slice of the agent’s environment state that covers all heterogeneous resources that the agent mutates, PLANARIAN proposes a consistent snapshot and rollback/fork protocol on live, running sandboxes and services. At the start of PLANARIAN’s snapshot, the statepoint manager first freezes the container’s processes via cgroup freeze. It then takes the CRIU checkpoint and ZFS snapshot, and captures the remote state through the state proxy. The container’s processes are unfrozen once all captures finish, ensuring that the sandbox state does not drift and thus stays consistent with the other artifacts. The statepoint manager only marks a statepoint as committed once all captures succeed; otherwise the statepoint manager marks it as pending and does not treat it as a valid statepoint. Rollback restores them as a single operation.

The above mechanism lets PLANARIAN maintain a stop-the-world invariant on the container across the entire snapshot window, including CRIU’s internal process dump phase, where the processes remain frozen after the dump and resume only when PLANARIAN explicitly unfreezes the cgroup.

This lets PLANARIAN freeze the container activity during the snapshot without stopping the Podman container, avoiding the cold-start overhead on every snapshot operation.

As PLANARIAN cannot quiesce a running external service that does not expose such APIs, PLANARIAN’s statepoint manager takes snapshots between tool calls. As each MCP server exposes a single service, one tool call typically mutates only one external service. Therefore, under the assumption that the external service is logically isolated by tenant (i.e., the agent-readable and agent-mutable state of the external service is not affected by other tenants), PLANARIAN guarantees consistency across the sandbox and the external services.

On rollback, the statepoint manager first stops the container, then applies the compensating actions via the state proxy and restores the file system. Once all other resources have been rolled back, CRIU starts its restore, making sure the container does not drift during the restore. Fork follows a similar pattern, but if the remote state is only compensable rather than forkable, PLANARIAN prevents the forked sandbox from using the state proxy, which would otherwise cause a consistency issue between exploration branches.

6.3 Orchestrating snapshot and execution

Since the agent is waiting for the LLM’s response between tool calls, the state management operations can be run in parallel with the LLM inference request. PLANARIAN is designed as a lightweight, daemonless runtime on top of Podman, CRIU, and ZFS, without requiring a dedicated runtime service. To make both snapshot and fork asynchronous, PLANARIAN launches background workers and coordinates them using a per-sandbox file lock. Tool calls, including command execution and MCP requests, acquire a shared lock, enabling concurrent tool execution. Snapshot and rollback either need the state to be frozen or mutate it, so they acquire an exclusive lock, waiting for in-flight tool calls to finish and preventing new tool calls from executing until they complete. Since fork does not modify the source sandbox, it acquires a shared lock on the source sandbox and an exclusive lock on the forked sandbox, whose state it mutates. Snapshot and fork thus overlap with LLM inference while preserving statepoint consistency.

7 Evaluation

We evaluate PLANARIAN to answer the following questions: (i) How does PLANARIAN perform under agentic exploration tasks (§7.2)? (ii) What is PLANARIAN’s performance compared to existing state management approaches on system administration and coding tasks (§7.3)? (iii) What is PLANARIAN’s overhead compared to no state management (§7.4)? (iv) What is the latency of PLANARIAN’s state management primitives (§7.5)?

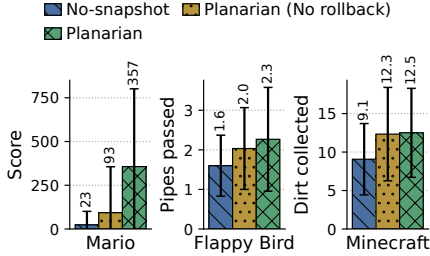


Figure 6. GameWorld, score

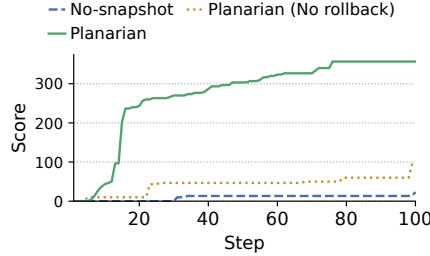


Figure 7. Mario, max score by step

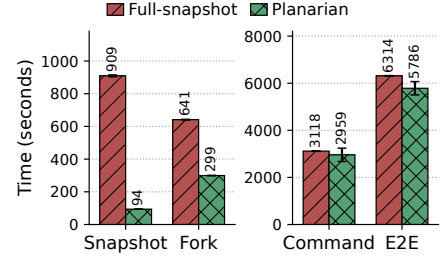


Figure 8. MCTS-style exploration

7.1 Experimental setup

Testbed. We evaluate PLANARIAN on an Ubuntu 22.04 server with Intel Xeon Silver 4310 CPUs (24 cores, 48 threads), 128 GB of memory, and NVMe SSD storage.

Implementation. We implement PLANARIAN in Go and Python. We use Podman 5.8.6 as the agent sandbox, ZFS 2.3.9 for sandbox storage, and CRIU 4.2.1 for process checkpointing. We also use the MCP toolbox for databases [6] as an MCP server. Podman’s container network is enabled when needed.

We integrate PLANARIAN with four agents: (i) Coding—we use mini-swe-agent [42], a lightweight software engineering agent designed for agentic coding tasks; (ii) Personal assistant—we employ nanobot [13], a Python implementation of openclaw [15] that automates everyday tasks; (iii) Exploration harness—we use SWE-search [19], an MCTS-style tree-search harness for coding tasks; and (iv) Gaming—we use the GameWorld [34] harness and benchmark.

Baselines. We compare PLANARIAN against two baselines: (i) NO-SNAPSHOT, which executes agent tasks in a Podman container without state management. (ii) FULL-SNAPSHOT, which runs PLANARIAN with Podman’s native snapshot mechanism that exports a full snapshot of the container’s process tree together with its file-system changes. Both baselines use an ext4 filesystem without CoW storage.

Agent workloads. We evaluate PLANARIAN with four real-world agentic workloads: (i) DevOps-Gym [39] consists of 704 DevOps tasks collected from over 30 projects; (ii) Terminal-Bench 2.0 [33] includes 89 tasks, spanning a wide range of categories from terminal environments; (iii) Bird-interact-lite [29] is an interactive SQL-generation benchmark with 300 database tasks; and (iv) GameWorld [34] is a gaming benchmark with 34 popular games.

7.2 Agentic exploration with state management

LLM-driven exploration. First, we measure how the LLM can proactively interact with PLANARIAN to explore the execution space and safely try different rollouts to find better execution plans in fewer steps. For this, we integrate PLANARIAN with GameWorld, enabling the gaming agent to roll back to a previous statepoint. When the game reaches a terminal state in a round, the LLM reviews all available

statepoints in the state ledger and either rolls back to a previous statepoint or resumes from the reset game state; after a rollback, the restore context is appended to the following prompts. The state ledger and restore context are constructed from the live game state of the GameWorld harness, with player-unobservable state filtered out.² Statepoints are captured between game plays, and only non-terminal states are available for rollback.

We evaluate PLANARIAN on the GameWorld [34] benchmark with three diverse games: Mario, Flappy Bird, and Minecraft. For each, we use the difficulty level at which the agent makes the most absolute progress toward the goal within the step limit. Note that the GameWorld-provided Minecraft game is a simplified item-collection simulator: to approximate the real game’s difficulty, we add real hazards to this simulator, including creepers, lava damage, etc.

As shown in Fig. 6, PLANARIAN enables the LLM to find a better strategy within 100 steps compared to no state management: the agent achieves 15 $\times$, 1.4 $\times$, and 1.4 $\times$ the NO-SNAPSHOT scores in Mario, Flappy Bird, and Minecraft, respectively. When the game enters a bad state (e.g., a reset due to death), the LLM uses PLANARIAN to roll back to a previous good state, explores alternative plans (as guided by the restore context), and achieves a higher score (see Fig. 7).

Note that snapshots may affect the timing of frames and may change the outcomes. To rule out this effect, we add a PLANARIAN (NO ROLLBACK) baseline, which uses PLANARIAN to snapshot after each game step but without exposing the rollback primitive. This variant achieves a higher score than NO-SNAPSHOT, but still lower than full PLANARIAN.

Harness-driven exploration. We integrate PLANARIAN with SWE-search [19], an MCTS-style tree search harness that explores the execution space of software-engineering tasks using Git-based versioning and branching. Since PLANARIAN supports snapshot/fork over the full sandbox state, we extend this harness to MCTS-style tree search on the Terminal-Bench tasks. Nodes in the search tree are snapshotted and forked on demand, overlapping these operations with LLM invocations. We use GPT-5.6-1una (medium reasoning) to generate MCTS-style tree-search traces, and replay the full traces, including the recorded LLM latency, command

²We define player-unobservable state as state that cannot be reconstructed from historical game screenshots.

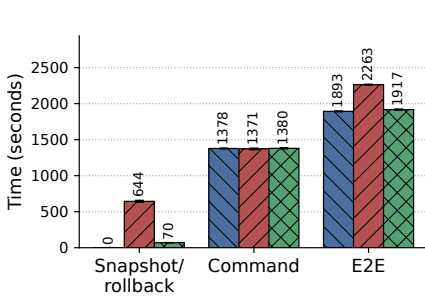


Figure 9. Terminal-Bench

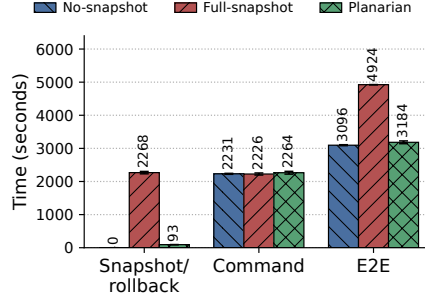


Figure 10. DevOps-Gym

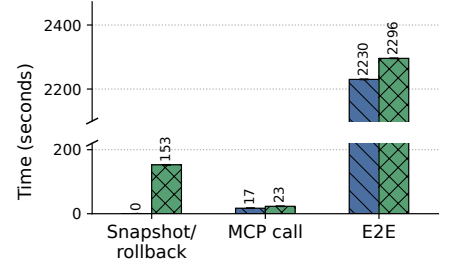


Figure 11. Bird-interact-lite

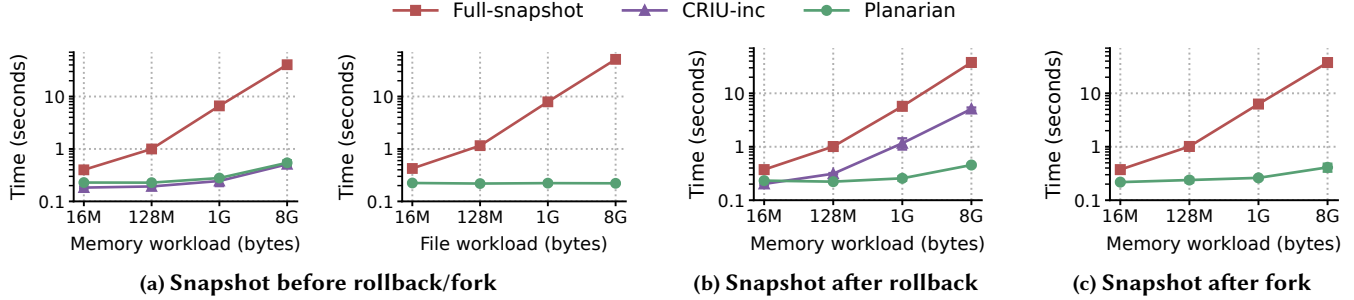


Figure 12. Snapshot cost

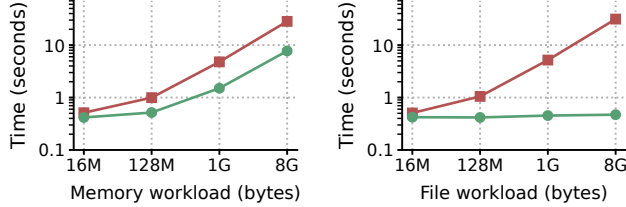


Figure 13. Rollback cost

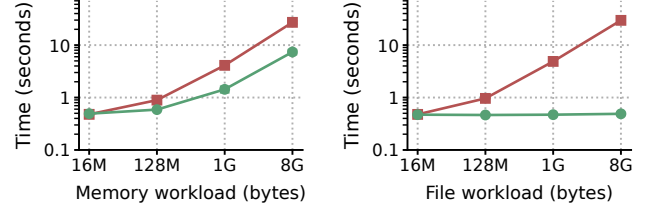


Figure 14. Fork cost

execution, and snapshot/fork operations on both PLANARIAN and FULL-SNAPSHOT. We compare PLANARIAN with FULL-SNAPSHOT: NO-SNAPSHOT is not included, because it lacks the fork primitive needed for MCTS-style search.

Fig. 8 shows the overhead of MCTS-style tree search with PLANARIAN and FULL-SNAPSHOT on nine system-administration tasks from Terminal-Bench, broken down into fork, snapshot, command execution, and end-to-end (E2E) time; the latter partially overlaps with snapshot and fork. FULL-SNAPSHOT’s primitives are significantly more expensive: its snapshot overhead is 10× that of PLANARIAN, because PLANARIAN checkpoints incrementally after a process restore, and its fork overhead is 2× that of PLANARIAN. As a result, FULL-SNAPSHOT is 9% slower than PLANARIAN in end-to-end time.

The end-to-end gap is smaller than the primitive gap, because FULL-SNAPSHOT also benefits from overlap: each node in SWE-search issues two LLM calls, and the resulting LLM latency is long enough to hide most of FULL-SNAPSHOT’s overhead. With more frequent snapshot/fork operations or with lower LLM latency, less of that overhead can be hidden, and the end-to-end gap grows toward the performance gap of the snapshot/fork primitives.

7.3 Safe execution with state management

We next evaluate PLANARIAN’s performance when used for user-driven rollback. We first use the PLANARIAN-integrated agent with GPT-5.6-luna (medium reasoning) to generate tool-calling traces, then replay these traces on PLANARIAN and the two baselines, because LLM outputs are non-deterministic.

Terminal. In this experiment, the mini-swe-agent runs on PLANARIAN and executes tasks from Terminal-Bench (nine tasks from the system-administration category). PLANARIAN snapshots the agent sandbox after each tool call, aligning with the granularity of human approval. At the end of execution, both PLANARIAN and FULL-SNAPSHOT roll back to a previous snapshot.

Fig. 9 shows that, compared with NO-SNAPSHOT, PLANARIAN adds a 1% overhead, which is significantly faster than FULL-SNAPSHOT (20%), because FULL-SNAPSHOT’s snapshots are more heavyweight. Compared with FULL-SNAPSHOT, PLANARIAN reduces raw snapshot/rollback times by 89%. Most Terminal-Bench tasks are not filesystem heavy, so copy-on-write does not add significant overhead.

DevOps. We integrate mini-swe-agent with PLANARIAN to evaluate performance on DevOps tasks. We use DevOps-Gym’s 16 implementation tasks in the build/configuration category. Fig. 10 shows that PLANARIAN requires less than 3% extra task completion time compared to NO-SNAPSHOT; FULL-SNAPSHOT adds a 59% overhead. Since FULL-SNAPSHOT’s sandbox snapshot/rollback time is $\sim 24\times$ that of PLANARIAN, it becomes challenging to hide the snapshot time behind the LLM inference time.

7.4 Overhead of consistent state management

To evaluate PLANARIAN’s overhead when both local and remote state are managed consistently, we execute 50 database management tasks from the Bird-interact-lite [29] benchmark, in which an agent mutates a PostgreSQL database via MCP [6]. For PLANARIAN, the state proxy generates compensating actions and rewrites SQL requests. We use GPT-5.6-luna (medium reasoning) to generate traces for MCP requests. As in §7.3, we then replay the requests on both NO-SNAPSHOT and PLANARIAN, with PLANARIAN snapshotting after each request and rolling back the sandbox and database.

Fig. 11 shows that, while managing local and remote state consistently, PLANARIAN adds a 3% overhead to the end-to-end time. This overhead comes from three sources: (i) when interacting with the MCP database server, the harness sometimes sends several requests instead of interleaving them with LLM inference, preventing snapshot time from being hidden; (ii) PLANARIAN incurs extra rollback operations for both the sandbox and database state; and (iii) to make MCP requests revertible, the state proxy adds SQL operations: PLANARIAN takes $1.3\times$ the MCP round-trip time compared to the NO-SNAPSHOT baseline. Since agentic database operations and MCP requests have low latency, this request amplification only increases end-to-end time by less than 0.3%.

7.5 State management primitives

We also evaluate the overhead of PLANARIAN’s state management primitives under different microbenchmarks. We use a workload that dirties memory or the file system. We take two successive snapshots and measure the overhead of the second, which is incremental when supported. We then measure the overhead of rolling back to, or forking from, this second statepoint. For the memory workload, we also measure the latency of the first snapshot taken after the rollback and after the fork. We compare PLANARIAN with FULL-SNAPSHOT and with CRIU-INC, which wires CRIU’s native incremental checkpointing through Podman.

Fig. 12 shows the overhead under these workloads. With a large working set (8 GB), PLANARIAN’s incremental snapshot achieves $74\times$ and $231\times$ speedups over FULL-SNAPSHOT on the memory and file system workloads, respectively, because FULL-SNAPSHOT always provides a full dump. PLANARIAN’s continuous incremental checkpoint protocol is also faster than CRIU’s native incremental checkpointing for the first

snapshot after rollback: on the 8 GB memory workload, PLANARIAN provides an $11\times$ speedup over CRIU-INC, because PLANARIAN continues to checkpoint incrementally after a restore.

We also measure the overhead of the rollback and fork primitives on the incremental snapshot (see Figs. 13 and 14). For rollback, PLANARIAN achieves $4\times$ and $67\times$ speedups over FULL-SNAPSHOT on the 8 GB memory and file system workloads, respectively. Restoring the filesystem is faster than restoring memory, due to the cost of reconstructing a large process. Fork shows a similar pattern: for a sandbox with the same memory and file system state, PLANARIAN exhibits $4\times$ and $61\times$ speedups over the same baseline. We conclude that, with the incremental checkpoint protocol and CoW storage, PLANARIAN provides efficient state management primitives.

8 Related Work

State recovery for agents. Existing systems version the state an agent mutates, either to recover the system from the agent’s mistakes or to make the agent’s state portable across machines. AgentFS [40] is a SQLite-backed overlay file system that makes the agent’s file system state checkpointable and portable. It covers file system state only, and a checkpoint copies the entire writable overlay layer. Daytona [9] and E2B [12] are cloud sandbox providers that offer persistent sandbox snapshots from which new sandboxes are created. They focus on the sandbox state itself, leaving remote state out of scope. Similarly, Agent Sandbox [32] supports persistent, stateful execution of Kubernetes (k8s) pods, so that idle agent sandboxes can be paused. k8s is designed to host remotely deployed microservices, whereas PLANARIAN targets the user’s local deployment; PLANARIAN can integrate with Agent Sandbox to make such remotely deployed sandboxes recoverable.

Other work makes external database state recoverable. Dolt [11] provides Git-like operations on database state, enabling users to revert, branch, and merge the agent-mutated database. Similarly, Neon [14] provides a CoW branching primitive for PostgreSQL. Both branchable database services can be integrated with PLANARIAN through its hooks, enabling consistent rollback and branching across both local and remote state.

Agentic explorations. SWE-search [19] is an MCTS-style search that executes, evaluates, and expands high-reward nodes, enabling search over Git-based software engineering tasks. PLANARIAN extends this beyond Git, enabling tree search on arbitrary terminal tasks. ExACT [44] provides LLM-driven MCTS exploration for GUI tasks and relies on backtracking in the web environment. PLANARIAN can integrate with ExACT, enabling backtracking even when the workload does not expose this capability.

Snapshot/restore systems. Various snapshot and restore systems support capturing a live system state and rollback

when needed. Checkpoint/Restore In Userspace (CRIU) [7] is a userspace process checkpointing tool that captures the state of a process tree, including memory pages, active sockets, and file descriptors, into a restorable image. As an alternative, DMTCP [18] is a process checkpointing library that dumps process state by preloading its library into the application at launch. We use CRIU for process checkpointing in PLANARIAN since it is widely integrated with OCI runtimes. Snapshots are also natively supported by many file systems and device mappers. The Zettabyte File System (ZFS) [17] is a CoW file system that pools the available storage and manages the pool state as a Merkle tree. Since it provides block-level CoW, PLANARIAN integrates ZFS for fast file system snapshots. Other CoW snapshot mechanisms for persistent state include Btrfs [35], a Linux-native file system that provides CoW over a B-tree, and dm-snapshot [10], a kernel device mapper that provides fast snapshots of a block device.

9 Conclusions

We presented PLANARIAN, an agent runtime that manages local and remote environment state through *agent statepoints*: consistent, restorable point-in-time versions of the environment. Three primitives—*snapshot*, *rollback*, and *fork*—give users, harnesses, and agents a unified interface for recovery and exploration, while the context library keeps agents informed of restored state and past outcomes. Our evaluation shows that PLANARIAN improves task quality by enabling agents to explore alternatives and lets users recover from erroneous agent actions with little overhead.

References

- [1] 2017. CRIU Issue #401. <https://github.com/checkpoint-restore/criu/issues/401>. Accessed: 2026-08-26.
- [2] 2024. Model Context Protocol. <https://modelcontextprotocol.io>. Accessed: 2026-6-2.
- [3] 2025. Claude Code. <https://www.anthropic.com/product/claude-code>. Accessed: 2026-6-2.
- [4] 2025. Codex. <https://openai.com/codex/>. Accessed: 2026-6-2.
- [5] 2025. Langchain. <https://github.com/langchain-ai/langchain>. Accessed: 2026-6-2.
- [6] 2025. MCP Toolbox for Databases. <https://github.com/googleapis/mcp-toolbox>. Accessed: 2026-6-2.
- [7] 2026. CRIU: Checkpoint/Restore In Userspace. <https://criu.org>. Accessed: 2026-08-26.
- [8] 2026. Cube Sandbox. <https://github.com/tencentcloud/CubeSandbox>. Accessed: 2026-6-2.
- [9] 2026. Daytona documentation: Sandboxes and Snapshots. <https://www.daytona.io/docs/en/>. Accessed: 2026-09-19.
- [10] 2026. Device-mapper snapshot support. <https://docs.kernel.org/admin-guide/device-mapper/snapshot.html>. Accessed: 2026-09-19.
- [11] 2026. DoltHub documentation: what is Dolt? <https://www.dolthub.com/docs/introduction/what-is-dolt/>. Accessed: 2026-09-19.
- [12] 2026. E2B. <https://e2b.dev/>. Accessed: 2026-6-25.
- [13] 2026. Nanobot. <https://github.com/HKUDS/nanobot>. Accessed: 2026-6-2.
- [14] 2026. Neon documentation. <https://neon.com/docs/introduction>. Accessed: 2026-09-19.
- [15] 2026. OpenClaw. <https://github.com/openclaw/openclaw>. Accessed: 2026-6-2.
- [16] 2026. Podman. <https://podman.io/>. Accessed: 2026-6-2.
- [17] Matt Ahrens, Jeff Bonwick, Val Henson, Mark Maybee, and Mark Shellenbaum. 2003. The zettabyte file system. In *Proc. of the 2nd Usenix Conference on File and Storage Technologies*.
- [18] Jason Ansel, Kapil Arya, and Gene Cooperman. 2009. DMTCP: Transparent Checkpointing for Cluster Computations and the Desktop. In *Proc. of the IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*.
- [19] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2025. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement. In *International Conference on Learning Representations*.
- [20] Mahesh Balakrishnan, Ashwin Bharambe, Davide Testuggine, David Geraghty, David Mao, Vidhya Venkat, Ilya Mironov, Rithesh Baradi, Gayathri Aiyer, and Victoria Dudin. 2026. LogAct: Enabling Agentic Reliability via Shared Logs. *arXiv preprint arXiv:2604.07988* (2026).
- [21] Edward Y. Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 4874–4886. doi:10.14778/3750601.3750611
- [22] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [23] Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. *arXiv preprint arXiv:2606.17573* (2026).
- [24] Yunpeng Dong, Jingkai He, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Yubin Xia, and Haibo Chen. 2026. DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback. *arXiv preprint arXiv:2605.22781* (2026).
- [25] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.
- [26] Aaron Fanoos, Jacob Goldberg, Ank Agarwal, Joanna Lin, Anson Zhou, Sonnet Xu, Vasiliki Bikia, Roxana Daneshjou, and Sanmi Koyejo. 2025. Syceval: Evaluating llm sycophancy. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, Vol. 8. 893–900.
- [27] Xingyuan Hua, Sheng Yue, and Ju Ren. 2026. Learning to Explore: Scaling Agentic Reasoning via Exploration-Aware Policy Optimization. *arXiv preprint arXiv:2605.08978* (2026).
- [28] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM transactions on information systems* 43, 2 (2025), 1–55.
- [29] Nan Huo, Xiaohan Xu, Jinyang Li, Per Jacobsson, Shippei Lin, Bowen Qin, Binyuan Hui, Xiaolong Li, Ge Qu, Shuzheng Si, Linheng Han, Edward Alexander, Xintong Zhu, Rui Qin, Ruihan Yu, Yiyao Jin, Feige Zhou, Weihao Zhong, Yun Chen, Hongyu Liu, Chenhao Ma, Fatma Özcan, Yannis Papakonstantinou, and Reynold Cheng. 2026. BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation via Lens of Dynamic Interactions. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=nHrYBGujps>
- [30] Taesoo Kim, Xi Wang, Nickolai Zeldovich, and M Frans Kaashoek. 2010. Intrusion recovery using selective re-execution. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*.
- [31] Ákos Kovács. 2017. Comparison of different Linux containers. In *2017 40th International Conference on Telecommunications and Signal*

- Processing (TSP)*. IEEE, 47–51.
- [32] Kubernetes SIGs. 2026. Agent Sandbox: isolated, stateful, singleton workloads on Kubernetes. <https://github.com/kubernetes-sigs/agent-sandbox>. Accessed: 2026-09-19.
 - [33] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868* (2026).
 - [34] Mingyu Ouyang, Siyuan Hu, Kevin Qinghong Lin, Hwee Tou Ng, and Mike Zheng Shou. 2026. Gameworld: Towards standardized and verifiable evaluation of multimodal game agents. In *European Conference on Computer Vision*. Springer, 41–59.
 - [35] Ohad Rodeh, Josef Bacik, and Chris Mason. 2013. BTRFS: The Linux B-tree filesystem. *ACM Transactions on Storage (TOS)* 9, 3 (2013), 1–32.
 - [36] Rui Shu, Peipei Wang, Sigmund A Gorski III, Benjamin Andow, Adwait Nadkarni, Luke Deshotels, Jason Gionta, William Enck, and Xiaohui Gu. 2016. A study of security isolation techniques. *ACM Computing Surveys (CSUR)* 49, 3 (2016), 1–37.
 - [37] Ya-Yunn Su, Mona Attariyan, and Jason Flinn. 2007. AutoBash: improving configuration management with operating system causality analysis. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles* (Stevenson, Washington, USA) (SOSP '07). Association for Computing Machinery, New York, NY, USA, 237–250. doi:10.1145/1294261.1294284
 - [38] Maciej Świechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mańdziuk. 2023. Monte Carlo tree search: A review of recent modifications and applications. *Artificial Intelligence Review* 56, 3 (2023), 2497–2562.
 - [39] Yuheng Tang, Kaijie Zhu, Bonan Ruan, Chuqi Zhang, Michael Yang, Hongwei Li, Suyue Guo, Tianneng Shi, Zekun Li, Christopher Kruegel, et al. 2026. DevOps-Gym: Benchmarking AI Agents in Software DevOps Cycle. *arXiv preprint arXiv:2601.20882* (2026).
 - [40] Turso. 2026. AgentFS: The filesystem for agents. <https://github.com/tursodatabase/agentfs>. Accessed: 2026-09-19.
 - [41] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).
 - [42] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://arxiv.org/abs/2405.15793>
 - [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
 - [44] Xiao Yu, Baolin Peng, Vineeth Vajipey, Hao Cheng, Michel Galley, Jianfeng Gao, and Zhou Yu. 2025. Exact: Teaching ai agents to explore with reflective-mcts and exploratory learning. In *International Conference on Learning Representations*.
 - [45] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 1187–1204.
 - [46] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. 2026. ACR-Fence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore. *arXiv preprint arXiv:2603.20625* (2026).